

Reconciling Time Slice Conflicts of Virtual Machines With Dual Time Slice for Clouds

Taeklim Kim^{ID}, Chang Hyun Park^{ID}, Jaehyuk Huh^{ID}, *Member, IEEE*, and Jeongseob Ahn^{ID}

Abstract—The proliferation of system virtualization poses a new challenge for the coarse-grained time sharing techniques for consolidation, since operating systems are running on virtual CPUs. The current system stack was designed under the assumption that operating systems can seize CPU resources at any moment. However, for the guest operating system on a virtual machine (VM), such assumption cannot be guaranteed, since virtual CPUs of VMs share a limited number of physical cores. Due to the time-sharing of physical cores, the execution of a virtual CPU is not contiguous, with a gap between the virtual and real time spaces. Such a virtual time discontinuity problem leads to significant inefficiency for lock and interrupt handling, which rely on the immediate availability of CPUs whenever the operating system requires computation. To reduce scheduling latencies of virtual CPUs, shortening time slices can be a straightforward strategy, but it may lead to the increased overhead of context switching costs across virtual machines for some workloads. It is challenging to determine a single time slice to satisfy all the VMs. In this article, we propose to have dual time slice to resolve the time slice conflict problem occurred in different types of virtual machines.

Index Terms—Virtualization, virtual time discontinuity, dual time slice

1 INTRODUCTION

ONE of the changes by system virtualization is that operating systems are running on virtual CPUs (vCPUs) which time-share physical CPUs. Guest operating systems schedule their user and kernel threads, and are often interrupted by exception handling, under the assumption that the virtualized CPUs are continuously running. However, the vCPU time space for a virtual machine (VM) is in fact a sequence of discrete time slices in real time space. We call this the virtual time discontinuity problem.

Traditionally, the system software stack has been designed with the assumption that the operating system can seize the control of CPUs at any moment. Due to the assumption, the virtual time discontinuity problem can potentially incur significant inefficiency in two critical mechanisms which the current operating systems rely on for synchronization and interrupt handling. A simple and low overhead spin lock mechanism for synchronization is widely used in kernels since the critical sections of kernel threads are very short. However, with the virtual time discontinuity problem, the vCPU running the lock holder thread can be preempted by the hypervisor, making lock waiters spin unnecessarily, wasting CPU resources. The problem also incurs the reduction of I/O throughput and the slowdown of inter-processor interrupts

(IPIs). The fundamental assumption of the interrupt mechanism is that the CPU is readily available to serve incoming interrupts.

There have been several advancements in hypervisor schedulers and architectural support to address the two problems individually. For example, the popular Xen hypervisor boosts the priority of a vCPU if a virtual interrupt is pending for the vCPU, preempting a running vCPU. Furthermore, the PLE (Pause-Loop Exiting) support in the x86 architecture allows the hypervisor to detect the physical CPU being wasted due to the lock holder preemption, providing the physical CPU to another ready vCPU. However, such scheduling or architectural solutions address only a small portion of the virtual time discontinuity problem.

This paper investigates the virtual time discontinuity problem, which encompasses both delayed lock processing and interrupt handling. Based on the observation, this paper identifies the possibility of shortening the time slice for vCPU scheduling to sub-millisecond ranges. By reducing the time slice to a more fine-grained time quantum of less than 1ms, the scheduling artifact of the current time multiplexing of vCPUs in the range of tens of milliseconds can be hidden. By multiplexing vCPUs with such a short time slice, the scheduling turn of each vCPU arrives quickly without a significant delay. The critical section or interrupt handling can be completed within the short time period, commonly within 1ms. However, one of the most significant downside of shortening time slice is the overhead of frequent context switches. The applications, which highly exploit the shared last-level cache, would suffer from increased the number of context switches because the cache can be frequently and quickly polluted by other applications sharing the pCPUs.

Thus, a fixed time slice for all the VMs causes a conflict of interest for the length of time slices. This paper introduces

- T. Kim and J. Ahn are with the Department of Software and Computer Engineering, Ajou University, Suwon 16499, South Korea. E-mail: {limkim4233, jsahn}@ajou.ac.kr.
- C. H. Park is with the Department of Information Technology, Uppsala University, 752 36 Uppsala, Sweden. E-mail: chang.hyun.park@it.uu.se.
- J. Huh is with the School of Computing, Korea Advanced Institute of Science and Technology, Daejeon 34141, South Korea. E-mail: jhhuh@casys.kaist.ac.kr.

Manuscript received 25 Aug. 2019; revised 24 Apr. 2020; accepted 30 Apr. 2020.

Date of publication 8 May 2020; date of current version 21 May 2020.

(Corresponding author: Jeongseob Ahn.)

Recommended for acceptance by J. Zhan.

Digital Object Identifier no. 10.1109/TPDS.2020.2993252

operating dual time slice (short and long) which can serve various applications which do not have the same preference for the length of time slices. To have dual time slice in clouds or datacenters, we allow physical servers to support either short or long time slice CPU pool, instead of a fixed time slice. Also, we enable the time slice of CPU pools to be adjustable at runtime. For applications which are significantly affected by the virtual time discontinuity problem, we place them on the short time slice pool to be frequently and quickly scheduled. This can reduce the waiting time to acquire the spin lock and to handle interrupts. With a long time slice pool, we can preserve performance for cache-sensitive applications that are affected by frequent context switching rather than the virtual time discontinuity. To place VMs into the appropriate time slice group, we profile the performance behavior of VMs at runtime to identify the suitable time slice of each VM, and coordinate VM migration within and across machines for all VMs that need to be moved to different time slice groups.

The new contributions of this paper are as follows. First, this paper addresses the virtual time discontinuity problem in a holistic manner. Prior studies approach spin locks, external interrupts, or IPIs in VMs as an individual problem. This paper generalizes them into the virtual time discontinuity problem. Second, this paper emphasizes the importance of reducing the time slice to resolve virtual time discontinuity. This study conducts a detailed analysis of sub-millisecond time slicing to understand the strengths and weaknesses of the short time slice. By varying the length of time slice and consolidation scenarios, we can figure out the cache pollution cost which is a major downside when reducing the time slice. Third, this paper proposes a virtual machine coordination scheme with dual time slice to resolve the time slice conflict problem among different types of virtual machines running on a single time slice.

Our real machine evaluation shows that decreasing time slice can mitigate the virtual time discontinuity problem effectively both for lock and interrupt-intensive VMs. On the other hands, shortening time slice can degrade performance for certain workloads in SPEC CPU2006 [14] benchmark because cache pollution is exacerbated by the short time slice. With our dual time slice scheme, we can reconcile such time slice conflicts among VMs. We show performance speedup up to 80 percent from the various experiments that include real world workloads of CloudSuite [13] and GRAPH500 [6] while minimizing performance degradation for cache-sensitive applications.

2 MOTIVATION AND BACKGROUND

In this section, we introduce the virtual time discontinuity problem in virtualized systems, and present the background and current solutions for two affected kernel mechanisms, spin lock processing and interrupt handling by guest operating systems.

2.1 Virtual Time Discontinuity Problem

In virtualized systems, many VMs share a physical system. Virtual CPUs (vCPUs) of the VMs are multiplexed to physical CPUs by the hypervisor. When the number of active vCPUs exceeds the available pCPUs, vCPUs time-share pCPUs,

receiving their share of time slice periodically. The quantum of time slice for each scheduling round is coarse-grained, commonly several to tens of milliseconds, to amortize costs for context switching. The guest operating system assumes virtually continuous execution of its vCPUs, but in real time space, each vCPU is periodically scheduled in and out. In current virtualized systems, virtual machines are commonly over-committed to share physical resources more efficiently, and thus, virtual time discontinuity occurs frequently.

One critical problem of virtual time discontinuity is that the operating system is not aware of the discrete CPU execution slices. An important design assumption of current operating systems is that the operating system can seize the control of pCPUs immediately to process kernel operations or serve interrupt requests quickly. Two most important mechanisms that rely on this assumption are spinlock-based critical sections in kernel and handlers for external and inter-processor interrupts. The next two subsections describe the two problems in details.

2.2 Spin Lock in Kernel

The kernel uses spinlocks since they have low acquire latencies when the lock contention is low. Since the critical sections are commonly short in the kernel, kernel threads need to spin for only a short period of time without wasting CPU resources significantly in native systems. However, with virtual time discontinuity, the vCPU running a lock-holding thread can be scheduled out by the hypervisor, while lock waiting threads are spinning on the lock. The lock waiting threads can potentially waste CPU resources, until the vCPU running the lock holder thread is scheduled in again. This is the lock holder preemption (LHP) problem.

Architectural support for reducing LHP problem is available. Current x86 processors support the Pause-Loop Exiting (PLE) mechanism. Spin lock implementation in the guest kernel executes a pause instruction for each spin, and if a core detects that too many pause instructions are executed, exceeding a configurable threshold within a short period, a `vmexit` event is generated for the hypervisor. The hypervisor preempts the spinning vCPU, and schedule another ready vCPU, expecting the new vCPU to run useful instructions instead of spinning. Although PLE improves the throughput of the physical systems by reducing spins, it can impact the fairness across different VMs.

2.3 Interrupt Handling

Interrupt handling can also suffer from virtual time discontinuity. In non-virtualized systems, the interrupt is a mechanism for a HW device to send a request to the kernel to receive immediate services. An occurrence of interrupt immediately invokes the corresponding handler, with the assumption of the availability of CPUs by preemption. I/O throughput is affected by such interrupt handling efficiency. Furthermore, IPIs (inter-processor interrupts) for communication among CPUs also rely on the interrupt mechanism. For example, during a page table update and TLB shutdown process, IPIs are sent to invoke TLB flushes in the other processors.

The processing of an interrupt is relatively short-lived, and the user process is resumed quickly after interrupt

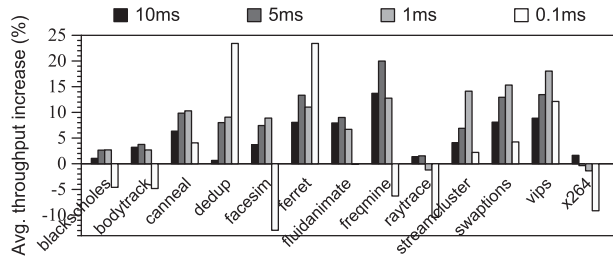


Fig. 1. Average of aggregate throughputs for all combinations of PARSEC: 10ms, 5ms, 1ms, and 0.1ms.

handling in native systems. However, in virtualized systems, the hypervisor sends a virtual interrupt to a VM, and if the vCPU is not scheduled, the virtual interrupt cannot be served immediately. The vCPU will be able to process the interrupt only when it receives its time slice again from the hypervisor scheduler, postponing the processing of urgent interrupts. Although the actual CPU processing time for interrupt handling is short, the scheduling latency will force a VM to delay interrupt handling by tens of milliseconds or longer, depending on how many vCPUs share a pCPU. Such a delay can significantly reduce the I/O throughput, and increase the latencies of TLB shutdown processes.

To mitigate the problem, the current Xen hypervisor optimizes the hypervisor scheduler for interrupts. When a vCPU, which is not running on a pCPU, receives a virtual interrupt, it will be immediately scheduled on a pCPU by preempting the currently running vCPU on that pCPU. Such a scheduling optimization mimics the behavior of interrupt handling in native systems, by scheduling a vCPU to a pCPU immediately.

3 MICRO-SLICED CONTEXT SWITCH

3.1 Reducing Scheduling Latencies

In this section, to reduce the gap between virtual and real time spaces, we evaluate reducing time slices for each VM to reduce the scheduling latency. The length of the time slice can directly affect the scheduling latency as shown in the following equation.

$$\text{scheduling_latency} = \text{time_slice} \times (\#vCPUs - 1). \quad (1)$$

The $\#vCPUs$ of equation 1 denotes the number of vCPUs queued in the given physical CPU's runqueue.

Reducing the time slice alleviates the previously mentioned LHP and I/O problems. The preempted lock holder will be scheduled earlier, and thus will release the lock sooner. Any busy waiting lock waiters will be able to acquire the lock sooner, or will be scheduled out after busy waiting for a shorter period of time. As for the interrupt handling, whenever an interrupt is issued to a VM, the vCPUs of the VM will be scheduled with a shorter delay. This will allow a more prompt handling of the interrupt, resulting in improved I/O performance.

On the other hand, the short time slice will be accompanied by overheads which are classifiable into two types. The first type is the direct cost incurred by context switches, and virtual mode extensions (VMX) mode switches such as VM-exits and VM-enters. The second type is the indirect cost, which is the pollution of architectural structures such as caches, TLBs,

TABLE 1
Non-Mixed and Mixed Workload Combinations

	vm-1	vm-2	vm-3	vm-4
non-mixed	ferret	vips	dedup	3x swaptions (1) streamcluster (1)
mixed	ferret iperf(1)	vips	dedup	3x swaptions (1) streamcluster (1)

branch predictors, etc. We provide results from the single thread applications that show performance degradation because of short time slice in Section 3.2.3. Also, to minimize the negative impacts, maintaining long time slice is necessary and it will be discussed in depth in Section 4.

3.2 Experimental Results

We evaluate the effects of shortening time slices with a real system equipped with two Intel Xeon processors. Dynamic voltage and frequency scaling, turbo boost, and hyper-threading were disabled for consistent performance evaluation. We use the Xen hypervisor, with the dom0 VM running Ubuntu. To minimize the interference by dom0, one CPU socket is dedicated to dom0 and the other socket is used to run guest VMs.

3.2.1 Single (Multi-Thread) Application VM Scenarios

We evaluate the overall performance by running each of the PARSEC3.0 [7] benchmark in two different VMs. The overcommit ratio is 2 to 1. For completeness, we run every combination of two PARSEC applications and executed them together, excluding cases with two same applications. Fig. 1 presents the average aggregated throughput improvements for each tenant benchmark, compared to the default time slice of 30ms. For example, the values for dedup are averages of aggregate throughput of dedup running with 12 other PARSEC benchmarks. For most cases, a short time slice of 1 ms improved the average throughput effectively. As for the case of 0.1 ms, we observe a large variation of results, with some workloads improving, and others decreasing. The performance increase of dedup with 0.1 ms is because the lock preemption problem is severe in the application with frequent TLB shutdowns. The reason for the performance drop with the 0.1ms slice is due to the cost incurred, both direct and indirect, by frequent context switching.

3.2.2 Multi-Application VM Scenarios

In this section, we evaluate two consolidation scenarios with multiple applications in a VM. Table 1 describes the workload combinations for the two scenarios. All workloads in the same cell run on the same VM, and the value inside the parenthesis indicates the number of threads for the application. In the non-mixed set, the workloads consist of only CPU-oriented workloads with VM-4 running multiple applications. In the mixed set, VM-1 has both CPU-oriented application (ferret) and I/O-oriented application (iperf). The default number of threads is four for the unspecified workloads, and each VM has 4 vCPUs. We vary the time slices to 30, 1, and 0.1 ms.

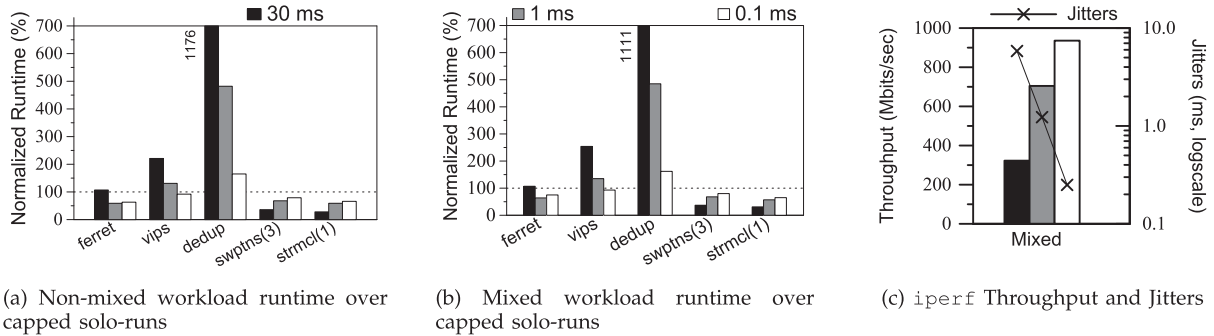


Fig. 2. Normalized execution time and I/O performance of multi-application VM scenarios.

The execution times are normalized to those with equal-capped solo runs. In the equal-capped solo runs, one VM runs on a physical system, but the CPU share is equal to the consolidated configuration in Table 1. The solo-runs provide the same amount of CPU to a VM as the consolidated runs when each VM receives the same share of CPUs, but they do not have the negative effect of time-sharing. In the setup, four vCPUs share a physical core, and in the equal-capped solo runs, a quarter of the total CPU share is allocated to the VM.

Non-Mixed Set. Fig. 2a shows the execution times with the non-mixed workloads, normalized to those with the equal-capped solo runs. Note that the parenthesis in figures indicates the number of executed benchmarks. swptns(3) indicates the average of three concurrent executions of single-threaded swaptions.

With the 30 ms time slice, the execution times increase by more than 10 times in dedup compared to the capped solo-runs. Even if the VM with dedup in both consolidated and capped solo runs can use the same CPU share, the negative effect on lock handling degrades the performance of dedup significantly. ferret and vips also suffer significantly by the artifact of time-sharing. However, swaptions and streamcluster improve the performance by time-sharing, since they receive more CPU share taken from the other applications. By decreasing the time slice to 1ms or 0.1 ms, the execution times of ferret, vips, and dedup are reduced significantly. However, the performance of the other two applications degrade since they can no longer take more CPU share from the other three applications.

From the perspective of fairness, reducing the time slice makes the performance of all VMs closer to the capped solo-runs. Such improvement of fairness is another benefit of short time slices in addition to throughput improvement. Furthermore, with 0.1ms time slice, all VMs exhibit lower execution times compared to the capped solo-runs except for dedup. dedup shows considerable performance improvement as well; not to the point where total fairness is achieved, but improving the overall system throughput nonetheless. *Mixed set.* Fig. 2b presents the normalized execution times for CPU-oriented applications with three different time slices. The results are in general similar to those in the non-mixed results. The additional application in this case is iperf, and Fig. 2c shows the I/O throughput and jitters with iperf as the time slice is reduced. With the 30ms time slice, iperf suffers from significant throughput degradation, because it executes on a VM with the CPU-intensive ferret application. In addition, the jitter exhibits a high

variance of 5.83 ms with the 30 ms slice. As the time slice is reduced, the VM running iperf and ferret is more frequently scheduled, resulting in iperf being scheduled more frequently. When the time slice is 0.1 ms, the throughput recovers to that of a solo-running iperf. Furthermore, jitters decrease to 1.23 with 1ms, and 0.25 with 0.1 ms.

Reducing the time slice of the scheduler can achieve both improved fairness and throughput on consolidated systems by eliminating the effect of virtual time discontinuity problem for lock and I/O-intensive workloads. However, such shortened time slices can often increase the context switching costs significantly.

3.2.3 Single-Thread Application VM Scenarios

To quantify the overhead which comes from frequent context switching, we select single thread applications of the SPECCPU2006 suite since these benchmarks are not affected by the virtual time discontinuity. We evaluate a couple of scenarios where each virtual machine runs multiple copies of single thread applications. For each VM, the number of application processes is equal to the number of virtual CPUs. The overcommit ratio is 2 to 1. We test five different benchmarks as tenants. Fig. 3 shows a histogram presenting performance variation for the time slice of 1, 5, and 10 ms, compared to baseline 30 ms. We also compare with a time slice of 90ms, which is longer than the baseline, to figure out how the long time slice could minimize the negative effects of frequent context switching.

In this evaluation, most of the single thread applications suffer from the short time slice. In particular, 1ms of Fig. 3a, the performance of xalancbm and gcc decrease by about 20 percent compared to the baseline 30ms. When a vCPU is scheduled to a pCPU, the private cache and parts of the LLC are filled with data fetched for the vCPU. The newly scheduled vCPU evicts the data used by the prior vCPU entirely. In current virtualized systems with coarse-grained time slices, the flushing effect has a minor impact on the virtual machine performance, since the time slice is long enough to amortize the effect of cache flushing. However, when the time slice is reduced, the phases in which cache contents are replaced occur more frequently, resulting in more cache misses. It is known that five benchmarks that are used as tenants use the LLC efficiently [3], so it results in lowered overall performance with short time slice. Note that namd and libquantum show that the performance degradation is negligible even in the 1ms time slice because

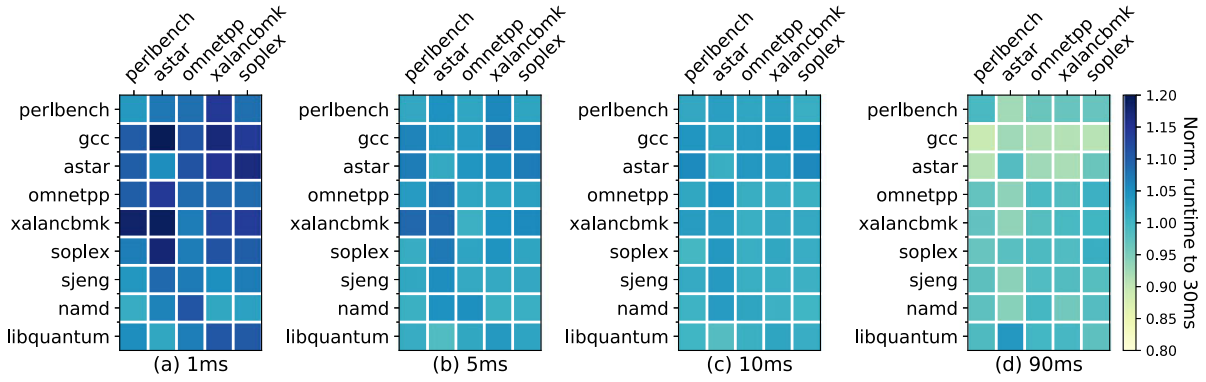


Fig. 3. Performance of each time slice for SPEC CPU2006 co-running with selected benchmarks.

these benchmarks are not sensitive to the capacity of the LLC. On the other hand, Fig. 3d presents the performance improvement of some benchmarks while taking longer time slice as 90 ms. For example, gcc and astar show better performance, because they are able to preserve the contents of the LLC for a bit long time. Except for those benchmarks, performance is slightly improved to the baseline.

To confirm the performance impact of the cache pollution in the reduced time slice, we conduct a simple experiment with hardware-assisted cache allocation technique introduced in recent commodity processors [15]. Table 2 compares performance of the shortened time slice with shared and isolated LLC for two VMs. Due to the limited space, we could not include the results for other workloads, but the phenomenon is similar to that. We configure the shared LLC as 6 MB and each VM is exclusively allocated 3 MB. The results are normalized to the 30 ms time slice without the cache isolation scheme. We observe that shortening time slice with cache isolation can alleviate the performance degradation by preserving the data on the limited cache capacity.

According to the experimental results, the shortened time slice (1 ms) becomes worse in the limited cache space for the cache-sensitive applications. The cache pollution is the major contributing factor to performance degradation with the 1 ms time slice. Thus, the long time slice (90 ms) is suitable for reducing the negative artifact of cache sharing.

4 DUAL TIME SLICE

As we have seen in the previous section, it is challenging to determine a single time slice to satisfy all the VMs because each VM has its own preferred time slice. To overcome the disadvantage of having a single time slice, we propose to have dual time slice across servers (or CPU sockets) to deal with both the virtual time discontinuity and cache pollution problems. As a set of physical CPUs can compose a CPU pool with its own time slice [36], we operate dual time slice on a single server as well as across multiple servers. VMs

that show similar performance behavior for the length of time slice map to the same type of CPU pools. Then the VMs are evenly distributed across the multiple pools of the same type to minimize load imbalance among the pools. On the other hand, we do not allow VMs, which have different features each other, to be placed on the same CPU pool. For example, VMs that are vulnerable to spinlock or IPI will not share the CPU pool with the VMs that are sensitive to LLC. By doing this, we can avoid the time slice conflict problem.

In this section, we introduce a simple but effective way of managing CPU pools in multiple server environments like clouds or datacenters. At scale, there are many possible ways to organize servers. Here, we advocate separating them into several groups, each of which has a different time slice. Fig. 4 depicts a baseline cloud organization that is comprised of a CPU (compute) pool and a storage pool (1). Since there is only a single CPU pool in the cloud, a fixed time slice is applied to all the VMs. On the other hand, we propose splitting a single CPU pool into multiple pools so that each different pool can have a different time slice that can be configured independently (2), allowing us to have time slice such as long and short. In such an environment, we place VMs highly affected by spinlocks, IPIs, or I/O interrupts on the CPU pools that are configured with short time slice, to alleviate the virtual time discontinuity problem. We can also benefit from other CPU pools with long time slice for cache-sensitive VMs. Since the running behavior of VMs can be changing over time, we need to be able to dynamically migrate VMs from one CPU pool with a short time slice to the other CPU pool with a long time slice or vice versa. This can be quickly done by changing the CPU pool affinity of a physical server. Furthermore, across

TABLE 2
Performance Impact of Cache Sharing and Isolation on 1 ms
(Normalized Execution Time to 30 ms)

	cache sharing	cache isolation
vm-1: gcc	1.10	1.06
vm-2: omnetpp	1.12	1.04

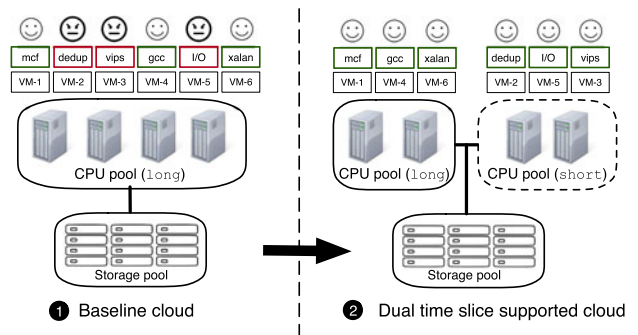


Fig. 4. Comparison of a baseline cloud with our proposed cloud organization.

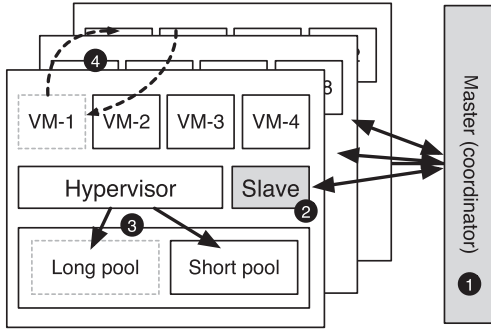


Fig. 5. Operating dual (short and long) time slice.

the servers, it can be achieved by VM live migration [9], which is supported by commodity hypervisors. We take the advantage of this mechanism to choose an appropriate CPU pool for each VM at runtime.

To balance the ratio between the short and long time sliced CPU pools, we allow the time slice of each CPU pool to be dynamically tuned. As a result, we do not need to decide the ratio statically beforehand. This can also minimize the number of VM live migrations incurring performance overheads because we can tune the time slice as desired if possible, instead of migrating VMs to the desired pool. This approach is reconciling the short time slice preferred VMs and the long time slice preferred VMs in multiple server environments.

4.1 Coordinating VMs With Dual Time Slice

We could require users to specify their preferred time slices because this is a definite way to satisfy the demands. In this study, however, we assume that users do not designate the preferred time slice as default. The goal of our scheme is to figure out preferred time slice of CPU pools for individual VMs and make groups of VMs, which have the same performance features in terms of the length of time slice. To find appropriate time slice of CPU pools for individual VMs, we need to monitor the running activities of VMs at a low cost. We monitor the events such as lock holder preemption and inter-processor interrupts that are related to virtual time discontinuity problems. In addition to that, we measure the MPKI (misses per kilo instruction) to estimate the slowdown of VMs due to contention in the last-level cache. We discuss the implementation details of our low-cost monitoring in Section 4.2.

Fig. 5 describes the overall architecture of the dual time slice. ① A master node periodically issues a request for collected events on all slave process in each compute node. Through monitoring those events, ② each compute node classifies whether the VMs are likely to be affected by either the virtual time discontinuity or the cache pollution problems. In the case of VMs that frequently experience the virtual time discontinuity problem, we consider them to be placed on the short time slice pool to minimize the CPU waiting time. If VMs are classified as cache-sensitive, those VMs are served on the long time slice pool to amortize the cache warmup overhead. For VMs which are not affected by the length of time slice, we allow them to be placed on either the short or the long time slice CPU pool flexibly. Once we complete the classification step, we try to make

groups of VMs based on the preferred time slices either short or long, called Intra-coordination (③).

It is not always possible to satisfy all the VMs to be placed in the preferred CPU pool in a single machine. For example, there are four VMs running on a server equipped with two CPU sockets so that we have three options configuring the two CPU pools. All the CPU pools can be configured either short or long time slice, or one CPU pool comes with the long time slice and the other CPU pool has the short time slice. Although the three configurations are able to meet diverse scenarios, it fails the case where one VM prefers the short time slice while rest of them show good performance with long time slice. In such a case, the time slice conflict cannot be resolved within a node while balancing the load across the CPU pools. It is required to find appropriate pools in other machines, called Inter-coordination (④).

Algorithm 1. Intra-Coordination

```

1: /* Profile and set time slice to each pool */
2: procedure IntraCoordination
3:   NUM_POOLS = 2 // number of NUMA nodes
4:
5:   /* (1) classify the type for each VM */
6:   for each  $vm_i$   $i$  in 1 ...  $n$  do
7:      $vm_i.stat = event\_collection(vm_i)$ 
8:      $vm_i.type = type\_classification(vm_i.stat_i)$ 
9:   end for
10:
11:   /* (2) adjust the time slice for each pool */
12:   for each  $pool_i$   $i$  in 1 ... NUM_POOLS do
13:     if not  $is\_tslice\_conflict(pool_i)$  then
14:       /* (2-a) VMs have the same preference */
15:        $set\_preferred\_tslice(pool_i)$ 
16:       goto out
17:     else
18:       /* (2-b) VMs don't have the same preference */
19:        $pool_i.conflict\_VMs = get\_mismatched\_VMs(pool_i)$ 
20:        $pool_i.inactive\_VMs = get\_inactive\_VMs(pool_i)$ 
21:        $pool_i.movable\_VMs =$ 
22:          $pool_i.conflict\_VMs + pool_i.inactive\_VMs$ 
23:     end if
24:   end for
25:
26:   /* (3) can resolve the conflict by exchanging VMs */
27:   if  $pool_1.movable\_VMs.cnt == pool_2.movable\_VMs.cnt$  then
28:      $migrate\_VM(pool_1.movable\_VMs, pool_2)$ 
29:      $migrate\_VM(pool_2.movable\_VMs, pool_1)$ 
30:   goto out
31:   end if
32:
33:   /* (4) cannot resolve the conflict in a machine */
34:   for each  $pool_i$   $i$  in 1 ... NUM_POOLS do
35:     if  $is\_tslice\_conflict(pool_i)$  then
36:        $set\_default\_tslice(pool_i)$ 
37:        $report\_to\_master(pool_i.conflict\_VMs)$ 
38:     end if
39:   end for
40: end procedure

```

Intra-Coordination. Algorithm 1 coordinates VMs with respect to the CPU pools. When deploying VMs in the cloud, we do not restrict the placement of the CPU pool. VMs can

be placed in any pool unless the user designates a pool. Each physical server periodically profiles the aforementioned events and determines the preferred time slice of each VM (line 5 to 8). Then, we look up each pool to check whether the VMs have the same preference, and set the preferred time slice (line 13 to 16). If the different types of VMs are placed on the same CPU pool, resulting in a time slice conflict (line 18 to 23), we try to resolve the conflict by migrating VMs to the preferred CPU pool (line 27 to 31). When migrating VMs, we consider the load balance among CPU pools because if we place VMs based only on the preferred time slice, certain CPU pools can be highly oversubscribed while other CPU pools are underutilized. If the preferred pool does not exist on the host machine, we set the default time slice (30 ms) to that pool. Then, we notify the master node to resolve this conflict by migrating VMs to another machine (line 34 to 39).

Algorithm 2. Inter-Coordination

```

1: /* Decide which VMs are required to be exchanged */
2: procedure InterCoordination
3:
4:   /* (1) list the conflict VMs from the entire hosts */
5:   for each  $host_i$   $i$  in  $1 \dots n$  do
6:      $host_i.conflict\_VMs = IntraCoordination(host_i)$ 
7:      $entire\_conflict\_VMs = host_i.conflict\_VMs$ 
8:   end for
9:
10:  /* (2) pick the conflict VMs to exchange each other */
11:  for each  $vm_i$   $i$  in  $entire\_conflict\_VMs$  do
12:    for each  $host_j$   $j$  in  $1 \dots n$  do
13:      if  $vm_i.host \neq host_j$  then
14:        if  $candidates$  then
15:           $candidates = find\_VMs(host_j.conflict\_VMs, not$ 
16:             $vm_i.type)$ 
17:           $exchange\_VMs(vm_i, candidates.head)$ 
18:          break
19:        end if
20:      end if
21:    end for
22:
23:  /* (3-a) figure out which node is idle or fully loaded */
24:  for each  $host_i$   $i$  in  $1 \dots n$  do
25:     $ImbalanceNodes.idle = is\_idle(host_i)$ 
26:     $ImbalanceNodes.full = is\_full(host_i)$ 
27:  end for
28:
29:  /* (3-b) exchange VMs to resolve load imbalance */
30:  if  $ImbalanceNodes$  then
31:     $make\_fair(ImbalanceNodes.idle, ImbalanceNodes.full)$ 
32:  end if
33: end procedure

```

Inter-Coordination. Algorithm 2 shows the pseudo code of how we resolve the time slice conflict with server migrations. Once the *IntraCoordination* function returns the list of conflict VMs for each node (line 5 to 8), we traverse the conflict list (*entire_conflict_VMs*) to find the remote CPU pool, which can provide the preferred time slice of conflict VMs (line 11). For a conflict VM, we try to find a candidate VM, which should be one of the conflict VMs in the other machines with a different preference

(line 12 to 14). Then, if we find the candidate, the two VMs are swapped by live migration (line 15 to 17). If it fails to find a candidate in a different node, we repeat these steps by iterating the remaining hosts in the system. Finally, we do load balancing for each node, by determining which node is idle or fully loaded, and those nodes will be exchanged with live migration (line 24 to 32).

4.2 Implementation

We implemented our prototype in the Xen 4.7 based environment by extending the existing software including *xenlight* library (*xl*), and Intel's RDT tool. We build a lightweight monitoring tool which runs on domain-0 of each host server. The CPU pool for domain-0 is distinguished from CPU pools of guest VMs so that this tool does not invoke any virtual time discontinuity problem. We collect the statistics of yield counts from each vCPU as used in our prior study [2]. This event frequently occurs when the guest VM suffers from LHP problem and waiting for IPI. The vCPUs of those VMs yield the pCPU because they could not utilize the pCPU anymore. Also, to measure MPKI (Misses Per Kilo Instructions) of the last-level cache, we referenced the open source software developed by Intel [17]. This monitoring tool collects these events periodically and decide the preferred time slice for each VM. Through the profiling phase (three seconds), we took the VM with MPKI larger than 10 as a cache-sensitive VM. We also considered that the VM incurring yields by PLEs or IPIs larger than 10,000 is being affected by the virtual time discontinuity problem. We empirically determined the thresholds for each event with our set of workloads. The threshold may need to be adjusted on a different hardware platform. Like prior studies [29], [41], we can open an interface for an administrator to control the threshold so that it can be tuned.

First, we filtered out the VMs that exceed the MPKI threshold and did not examine the yield counts from PLEs or IPIs for that VMs, because the performance of those VMs are more sensitive to the cache than the length of time slice. Second, for the remaining VMs, we observed at whether the number of yields by PLEs or IPIs surpasses the threshold or not. Last, we marked the VMs, which are not affected by either long or short time slices, if those VMs incur a very few number of the events. We allow this type of VMs to be placed any CPU pools because those are not vulnerable to any time slice. Based on the events monitored, we performed the intra coordination step as explained in Algorithm 1.

After completing the coordination of each CPU pools, we inform master node to resolve conflict VMs if it exists. At the master node, we run our Algorithm 2 and perform the coordination such as initiating live migrations of VMs between machines. While placing VMs, we consider the load balancing across the CPU pools by accounting the number of active vCPUs. If any CPU pool becomes idle, we redistribute VMs to prevent those VMs from overloading. The master node also takes care of load balancing across servers. If one server is idle, it informs to the master for further migration of VM. We plan to release the code in our final version.

4.3 Discussion

Time Slice Parameter. In this study, we show the effectiveness of our scheme when two types of CPU pools either short (1

TABLE 3
Workloads and Initial CPU Pool Placement Used in the Evaluation for Single Server

(a) Static workload mixes					(b) Dynamic workload mixes			
	Pool-1		Pool-2			Pool-1	vm-1	dedup → soplex → omnetpp → gmake
	vm-1	vm-2	vm-3	vm-4		Pool-2	vm-2	sjeng x 6
mix 1	dedup	sjeng	gmake	namd			vm-3	xalanc → vips → gmake → soplex
mix 2	graph500	soplex	pagerank	omnetpp			vm-4	sjeng x 6
mix 3	vips	omnet	dedup	milc				

	Pool-1	vm-1	dedup → gmake → sjeng → xalanc
		vm-2	sjeng → soplex → omnetpp → dedup
	Pool-2	vm-3	xalanc → vips → gmake → soplex
		vm-4	omnetpp → sjeng → libqu → dedup

ms) or long (90 ms) time slice are supported. To amortize the cost of cache warmup, we select the longer time slice (90ms) than the default one (30 ms). For the case where we could not choose either the short or long time slice due to the conflict, we configure the CPU pool with the default time slice.

We expect the types of CPU pools to be more diverse for a wide variety of workloads. For example, we have seen that dedup with the time slice of 0.1 ms outperforms that of 1ms. In such a case, our scheme can be easily extended with multiple time slices, not just with dual time slice.

Cache Contention. Although our policy consolidates cache-sensitive VMs on the long time slice pools which can mitigate the cache pollution problem, compared to the short time slice pools, the performance interference across the VMs sharing the pool still remains. To minimize performance degradation due to cache sharing, we can take the advantage of the resource partitioning facility which is recently introduced in commodity hardware [15]. For the CPU pool serving cache-sensitive VMs, we can completely isolate the cache space so that each VM is able to keep its context on that cache. However, it cannot fully utilize the cache capacity so that certain VMs experience performance degradation due to the lack of the cache space. We believe that the cache preservation scheme [3] discussed in our prior study can be adapted on top of this CPU pool scheme.

NUMA Affinity. Our proposed schemes are not integrated with the NUMA-aware memory allocation scheme because the workloads affected by the virtual time discontinuity did not show the sensitivity to the NUMA affinity. On the other hand, the applications significantly exercising memory would be affected if remote memory accesses are significant. We believe that our schemes can be extended to further consider the NUMA affinity for VMs running on the long pool. While migrating VMs between pools, we can try to minimize the use of remote memory [1].

5 EVALUATION

5.1 Experimental Setup

We mimic a cloud environment with four sever machines. Four machines are used for hosting VMs and the other one acts as a master node in our study. Each server has two Intel Xeon E5-2650 v4 processors (12 cores each) with 64 GB DRAM and machines are connected via 10 Gbit private network to isolate the network traffics. We run our experiment on the Xen 4.7, and it is modified for implementing our monitoring tool. The guest VMs run Ubuntu 14.04 with Linux kernel 4.4 and configured with 10 virtual CPUs and 8 GB memory each. We reserve 2 cores from each socket for domain-0 so that there are total 20 cores running VMs and the overcommit ratio is 2 to 1. To minimize the execution time variations, we turned off the simultaneous multithreading (SMT) and dynamic voltage frequency scaling (DVFS) features in BIOS. We selected a couple of workloads from MOSBENCH, PARSEC3.0, CloudSuite, GRAPH500 and SPECPCPU2006 to have a diverse workload set. Tables 3, 4 and 5 show the list of our workload mixes and the initial CPU pool placement. Note that our scheme (dual time slice) starts from the initial placement, but the VMs will dynamically change placement during the run. First, we run four VMs to evaluate our scheme on a single server without server migration (mix 1 to 5). While conducting the evaluation, we also mimic the real-world scenarios that users are connecting to the server in the cloud, run their applications, and leaving. For example, the VM-1 in the mix 4 sequentially executes dedup, soplex, omnetpp, and gmake workloads one by one. Since those applications have different sensitivity on the time slice, this evaluation can show the effectiveness and robustness of our scheme. Second, we evaluate our scheme on multiple servers to see how much performance improvement can be achieved by migrating VMs (mix 6 to 13).

TABLE 4
Workloads and Their Initial CPU Pool Placements Used in the Evaluation for Two Servers

	Node 1				Node 2			
	Pool-1		Pool-2		Pool-3		Pool-4	
	vm-1	vm-2	vm-3	vm-4	vm-5	vm-6	vm-7	vm-8
mix 6	dedup	sjeng	gmake	xalanc	gmake	milc	soplex	omnetpp
mix 7	dedup	namd	soplex	movielens	milc	omnetpp	soplex	gmake
mix 8	movielens	sjeng	gmake	soplex	omnetpp	milc	soplex	pagerank
mix 9	pagerank	namd	omnetpp	gmake	xalanc	milc	soplex	movielens
mix 10	omnetpp	soplex	milc	perlbench	xalanc	omnetpp	libq	soplex
mix 11	astar	libq	xalanc	omnetpp	soplex	gcc	omnetpp	soplex

TABLE 5
Workloads Used in the Evaluation for Four Servers

	Node 1			Node 2		Node 3		Node 4	
mix 12	gmake dedup	sjeng omnetpp	movielens gmake	namd soplex	pagerank omnetpp	sjeng gmake	vips gmake	namd milc	
mix 13	milc soplex	omnetpp gmake	gmake dedup	namd omnetpp	xalanc soplex	milc pagerank	dedup gmake	sjeng soplex	

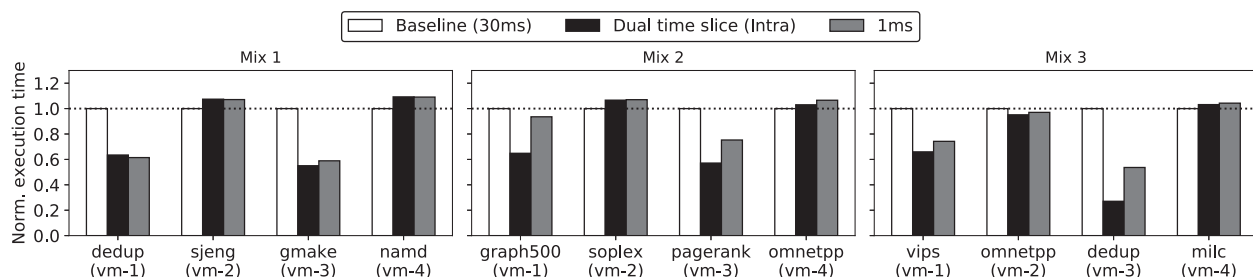


Fig. 6. Performance results for static workloads (mix 1, 2, and 3) with a single server.

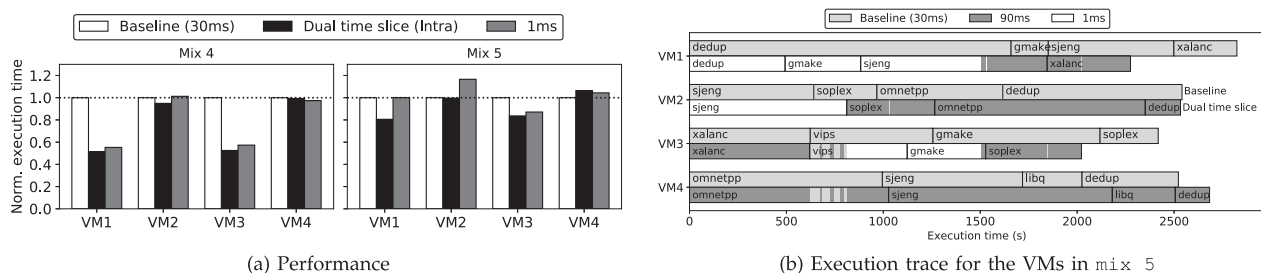


Fig. 7. Performance results for dynamic workloads (mix 4 and 5) with a single server.

5.2 Experimental Results

Performance Improvement With a Single Server. Fig. 6 shows the performance improvement when applying our dual time slice, compared to CPU pools with different time slices (90 ms and 1 ms). In this experiment, we use a single server to evaluate the effectiveness of the *intra* coordination scheme. For mix 1, 2 and 3, we use a variety of workload mixes where a single time slice incurs the conflict because *dedup*, *vips*, *pagerank*, *graph500* and *gmake* prefer to the 1ms time slice while *soplex*, *milc*, *omnetpp*, *namd* and *sjeng* do not benefit from the 1ms time slice. Since the parallel workloads including *dedup*, *vips*, *pagerank*, *graph500* and *gmake* can be highly affected by the virtual time discontinuity problem, the long time slice is not appropriate. On the other hand, *soplex*, *milc*, and *omnetpp* are cache-sensitive workloads so that the short time slice such as 1 ms degrades their performance due to the cache pollution problem. In our dual time slice, we maintain two different CPU pools, each of which is configured with the 90 ms time slice or 1 ms time slice respectively. For instance, in case of mix 2, we consolidate *graph500* and *pagerank* into the 1ms CPU pool while allowing *soplex* and *omnetpp* to run on the 90 ms pool. *graph500* and *pagerank* exhibit significant performance improvement, compared to both the baseline (30 ms) and 1ms, but the performance of *soplex* and *omnetpp* is slightly degraded. The main reason is that in the baseline, *soplex* and *omnetpp* were able to utilize the unused CPU cycles, which *graph500* or *pagerank* could

not use, according to the work-conserving policy [2], [3]. Once we serve *graph500* and *pagerank* in the 1 ms pool, they are able to have better CPU utilization than the default time slice. It results in that *soplex* and *omnetpp* cannot harvest the unused CPU cycles anymore.

Further, we also evaluate the case where the workloads running on VMs are changed as time goes by in Fig. 7a. The goal of this experiment is to figure out how well our dual time slice adapts to workload changes and we believe that these cases are common in datacenters or clouds. For mix 4 and mix 5, our scheme can achieve the best of both worlds, 90 ms and 1ms pools. To see the how our scheme works well, we trace the VM execution with respect to the type of time slices. Fig. 7b shows the execution time with the assigned CPU pools and time slices for each VM. For each VM, the upper gray bar indicates the execution time running on CPU pools with 30ms time slice as the baseline. The bar below shows the results of our scheme that the VMs dynamically assign to their preferred time slice CPU pool. We can clearly see that the completion time of a set of workloads running on each VM is reduced, compared to the baseline (30 ms). Specifically, the execution time of *dedup* of VM-1 and *gmake* of VM-3 can be significantly reduced by placing them on the 1ms CPU pool while serving *xalanc* and *soplex* on the 90 ms CPU pool. However, the execution time of VM-4 hosting *sjeng* is slightly increased. In the baseline (30 ms), *sjeng* of VM-4 could utilize the unused CPU cycles of VM-3 that serves *gmake* because *gmake* undergoes the virtual time discontinuity problem in the default

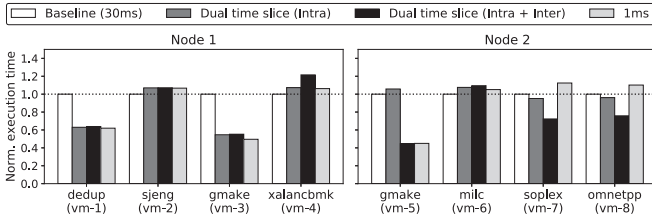


Fig. 8. Performance results for mix 6 with two servers.

30 ms time slice. On the other hand, with our scheme, *sjeng* runs with *soplex*. It means that both CPU intensive workloads could utilize the allocated CPU cycles well. As a result, the performance of *sjeng* slows down relative to the baseline.

Performance Improvement With Multiple Servers. Since we can fail to find appropriate CPU pools within a single server, we extend the search space of the CPU pools to remote servers. For this experiment, we performed two environments when the number of compute nodes is two and four. In addition to the compute server, we dedicate a server for the master node for each scenario.

For the two compute servers, the total number of CPU pools is four because each server has two CPU sockets. The workload used in this evaluation is presented in Table 4. Due to the limited space, we plot performance of individual VMs for mix 6 in Fig. 8 and for the rest of the workload mixes, we show the average speedup of VMs for each mix in Fig. 11. Fig. 8 shows how much performance improvement can be achieved with our scheme. In the baseline and 1ms configuration, each CPU socket serves two VMs and their placement is fixed as shown in the table. The dark gray bar presents the performance improvement with our intra coordination scheme and the black bar indicates the performance improvement with the combination of intra and inter schemes. We can see that most of the VMs in mix 6 shows improved the execution time. However, the performance of *sjeng*, *xalancbmk*, and *milc* is decreased. Fig. 9 presents the execution trace of the 8 VMs for mix 6 in the two servers. Initially, each node has two pools for each socket and they can be two types of CPU pools (90 ms or 1 ms). The execution time of individual VMs is plotted with an assigned CPU pool (color) and its time slice (hatch pattern). When analyzing the *gmake* of VM-3, it is running with *xalancbmk* initially while *dedup* and *sjeng* are placed on the other different pool with 1ms because *dedup* shows high number of yields from IPI. Although *gmake* prefers the 1ms time slice pool, our intra coordination scheme does not place the VM on the 1ms pool because *xalancbmk*, which is in the same pool, prefers longer time slice. This is a conflict of the time slice so we use the default (30 ms) time slice for the CPU pool (no pattern). While

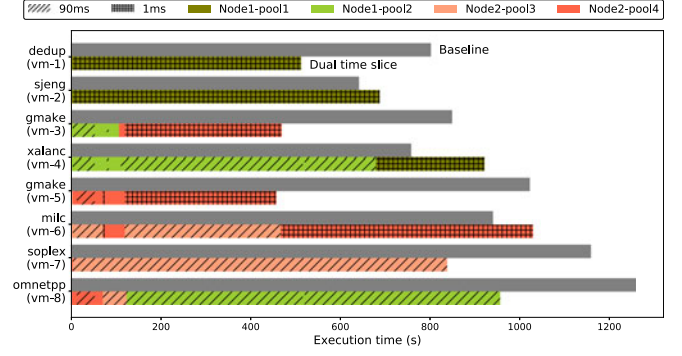


Fig. 9. Execution trace for the VMs in mix 6.

monitoring VMs on node 2, we observe that *gmake* of VM-5 is placed with *omnetpp* of VM-8 on the 30 ms pool. Since two *gmake* (vm-3 and vm-5) can be classified into the same type, and *xalancbmk* and *omnetpp* also show the similar performance characteristics, our inter coordination scheme can resolve this mismatch by swapping *gmake* of VM-3 and *omnetpp* of VM-8. After migrating two VMs, all 8 VMs could settle in their preferred time slice pool, and the execution time of two *gmake* reduced significantly. On the other hand, compared to the baseline, the performance of *xalancbmk* and *milc* is decreased. In the baseline, it runs with *gmake* so that it exercises the unused CPU cycles that *gmake* yields. However, with dual time slice, it no longer benefits from the co-runner. With the single time slice with 1ms, the parallel workloads can improve their performance, while the short time slice degrades the cache-sensitive VMs. On the other hand, with dual time slice, we can minimize the performance degradation observed in *soplex* and *omnetpp*.

To show the robustness of our scheme in the realistic cloud environment, we double the number of servers from two to four. Each server has the same configuration, two CPU pools with 10 physical CPU each, and the total number of VMs is 16. The workloads list we used appears in Table 5. In Fig. 10 of mix 12, our dual time slice enhances performance in most of the workloads from node 1 to node 4. Especially, the results from inter coordination scheme show better than intra scheme. For example, *dedup* of node 1 and *gmake* of node 2, 3 show better performance on inter than intra scheme, because it is possible to achieve their preferred time slice by utilizing live migration across servers. However, some workloads like *omnetpp* and *milc* degraded their performance, because they could not exploit the unused CPU cycles from *dedup* and *gmake*. Even so, the overall average speedup of mix 12 is 46 percent higher than the baseline and 27 percent better than the intra scheme as shown in Fig. 11.

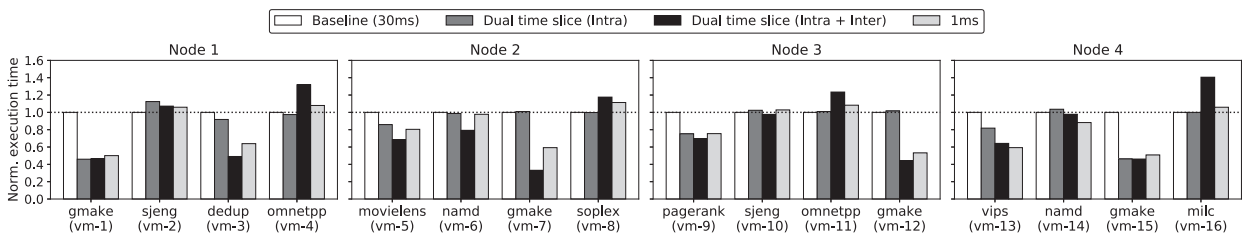


Fig. 10. Performance results for mix 12 with four servers.

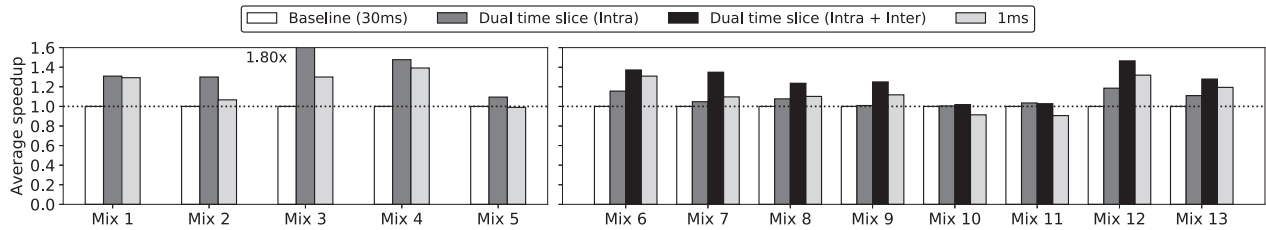


Fig. 11. Average speedup of the virtual machines for each mix.

Average Speedup Across all the VMs. Fig. 11 presents the average speedup across the VMs with the intra and inter schemes. Note that we evaluate on a single server from mix 1 to 5 so that it only includes the intra scheme. Results of mix 6 to 13 are evaluated on multiple servers, mix 12 and 13 are conducted on four nodes in particular. For all the workload mixes except for mix 10 and 11, the average speedup is around 4-80 percent with our dual time slice. With the introduction of our inter scheme, the performance can be further improved, compared to the intra only scheme. Although the performance improvement of the static 1ms configuration for mix 1 and 6 is noticeable, we demonstrate a negative impact of the static 1ms configuration on mix 10 and 11. For mix 10 and 11, we configure that all the workloads are cache-sensitive to see whether our scheme can cope with the situations that degrade in fixed 1 ms time slice. Since it is difficult to meet the time slice preference of VMs with a single time slice, our scheme dual time slice can show high performance in clouds that host a wide variety of workloads.

6 RELATED WORK

There have been many prior efforts to mitigate the artifact of virtualization including synchronization and interrupt handling by improving the hypervisor scheduler. Traditionally, co-scheduling provides a simple way to minimize the negative effects of CPU virtualization, running virtual CPU siblings at the same time [12], [34]. It can give an illusion to the guest operating system that their CPUs are dedicated to the VM. Although it can resolve the lock holder and waiter preemption problem, it causes CPU fragmentation problems, which lower the utilization of CPU resources to allow synchronized scheduling of sibling vCPUs. To minimize the negative effect, balanced scheduling was introduced [30]. It attempts to co-schedule the virtual CPU siblings, but it may occasionally break the strict requirement to avoid wasting CPU resources. However, the scalability of co-scheduling for many-core VMs is not fully investigated. Recently, for fast responses to IPI (inter-processor interrupts) demand-based scheduling boost the virtual CPU priority by capturing IPI signals [8], [11], [21], [28]. To mitigate the lock holder preemption problem, guest OS level approaches are intensively explored until recently [19], [20], [24], [27], [31], [32].

In addition, the interrupt processing delay was widely discussed by several prior studies [18], [22], [23], [26], [37], [38], [39]. vSlicer used different scheduling quanta for latency sensitive VMs and non-latency sensitive VMs for fast interrupt handling of latency sensitive VMs [39]. In vSlicer, VM users or the admin must assign such latency sensitive VMs manually. vTurbo addressed such mixed VM scenarios by using a dedicated core for interrupt

handling [38]. As mentioned before, these works focused on fixing each individual problem in an ad-hoc manner. Our approach is to look at the lock, IPI, and I/O related problems as a whole, and we proposed solving the virtual time discontinuity problem with the use of dual time slice. Teabe *et al.* studied a similar approach with ours, but they did not address the case where the time slice conflict cannot be resolved in a machine [32].

The architecture community also have discussed the spin-lock problems in virtualization. Spin Detection Buffer [35] is an architectural support mechanism to detect CPUs being wasted due to the lock holder preemption problems. The goal of Spin Detection Buffer is similar to the newly added Intel's PLE [16] and AMD's Pause Filter [4]. The AMD virtual interrupt controller (AVIC) [5] allows interrupt handling for APIC register r/w, IPIs, and I/O interrupts to bypass the hypervisor. Recently, to minimize the negative effects of cache sharing in overcommitted systems, cache restoration techniques were introduced [10], [40]. The cache contexts of the virtual machine being scheduled are restored by prefetching. In traditional operating systems, Mogul and Antia studied the effects of context switches incurring cache pollution in a single processor [25]. Tsafir explored the overhead of mode switch incurred by timer interrupts in operating systems [33].

This paper extends our prior study [3] with a new detailed analysis for cache-sensitive applications and introduces an effective way to find the best CPU pool in cloud-like environments.

7 CONCLUSION

This paper analyzed the performance behavior of virtual machines with regard to the length of time slice. Since it is difficult to satisfy all the virtual machines with a fixed time slice, this paper explored operating dual time slice to achieve the best of both worlds of short and long time slice. We coordinated the VMs, which cause the synchronization or interrupt handling issues, into the CPU pool with a short time slice while serving cache-sensitive VMs on a long time slice. In clouds, we could organize multiple servers into several groups, each of which has a different time slice. Our scheme determines the appropriate time slice for each VM at runtime and then reconciles the time slice conflicts by scheduling VMs. The evaluation showed that our scheme improves average performance among all VMs of cloud scenarios at a low cost.

ACKNOWLEDGMENTS

This work was supported by the Basic Science Research Program through the National Research Foundation of

Korea funded by the Ministry of Science and ICT (NRF-2013R1A2A2A01015514 and NRF-2019R1C1C1005166).

REFERENCES

- [1] J. Ahn, C. Kim, J. Han, Y.-R. Choi, and J. Huh, "Dynamic virtual machine scheduling in clouds for architectural shared resources," in *Proc. 4th USENIX Workshop Hot Topics Cloud Comput.*, 2012.
- [2] J. Ahn, C. H. Park, T. Heo, and J. Huh, "Accelerating critical OS services in virtualized systems with flexible micro-sliced cores," in *Proc. 13th EuroSys Conf.*, 2018, pp. 1–14.
- [3] J. Ahn, C. H. Park, and J. Huh, "Micro-sliced virtual processors to hide the effect of discontinuous CPU availability for consolidated systems," in *Proc. 47th Int. Symp. Microarchit.*, 2014, pp. 394–405.
- [4] "AMD64 architecture programmer's manual volume 2: System programming," *Programmer's Manual*, AMD, 2010.
- [5] "Introduction of AMD virtual interrupt controller," *Presentation Slides, XenSummit*, AMD, 2012. [Online]. Available: http://www.archive.xenproject.org/xensummit/xs12na_talks/M6b.html
- [6] J. Ang, B. Barrett, K. Wheeler, and R. Murphy, "Introduction to Graph500," Feb. 2020. [Online]. Available: <https://graph500.org>
- [7] C. Bienia, S. Kumar, J. P. Singh, and K. Li, "The PARSEC benchmark suite: characterization and architectural implications," in *Proc. 17th Int. Conf. Parallel Architectures Compilation Tech.*, 2008, pp. 72–81.
- [8] L. Cheng, J. Rao, and F. C. M. Lau, "vScale: Automatic and efficient processor scaling for SMP virtual machines," in *Proc. 11th Eur. Conf. Comput. Syst.*, 2016, pp. 1–14.
- [9] C. Clark *et al.*, "Live migration of virtual machines," in *Proc. 2nd Conf. Symp. Netw. Syst. Des. Implementation*, 2005.
- [10] D. Daly and H. Cain, "Cache restoration for highly partitioned virtualized systems," in *Proc. 18th Int. Symp. High Perform. Comput. Archit.*, 2012, pp. 1–10.
- [11] X. Ding, P. B. Gibbons, M. A. Kozuch, and J. Shan, "Gleaner: Mitigating the blocked-waiter wakeup problem for virtualized multicore applications," in *Proc. USENIX Annu. Tech. Conf.*, 2014, pp. 73–84.
- [12] D. G. Feitelson, "Distributed hierarchical control for parallel processing," *Computer*, vol. 23, no. 5, pp. 65–77, May 1990.
- [13] M. Ferdman *et al.*, "Clearing the clouds: A study of emerging scale-out workloads on modern hardware," in *Proc. 17th Int. Conf. Architectural Support Program. Lang. Operating Syst.*, 2012, pp. 37–48.
- [14] J. L. Henning, "SPEC CPU2006 benchmark descriptions," *SIGARCH Comput. Archit. News*, 2006.
- [15] A. Herdrich *et al.*, "Cache QoS: From concept to reality in the intel® xeon® processor E5-2600 v3 product family," in *Proc. 22nd Int. Symp. High Perform. Comput. Archit.*, 2016, pp. 657–668.
- [16] "Intel 64 and IA-32 Architectures Software Developer's Manual," *Software Developer's Manual*, Intel, 2010.
- [17] Intel, "Intel RDT Software Package," Jul. 2019. [Online]. Available: <https://github.com/intel/intel-cmt-cat>
- [18] W. Jia *et al.*, "Effectively mitigating I/O inactivity in vCPU scheduling," in *Proc. USENIX Annu. Tech. Conf.*, 2018, pp. 267–279.
- [19] S. Kashyap, C. Min, and T. Kim, "Scalability in the clouds!: A myth or reality?" in *Proc. 6th Asia-Pacific Workshop Syst.*, 2015, pp. 1–87.
- [20] S. Kashyap, C. Min, and T. Kim, "Scaling guest OS critical sections with eCS," in *Proc. USENIX Annu. Tech. Conf.*, 2018, pp. 159–171.
- [21] H. Kim, S. Kim, J. Jeong, J. Lee, and S. Maeng, "Demand-based coordinated scheduling for SMP VMs," in *Proc. 18th Int. Conf. Architectural Support Program. Lang. Operating Syst.*, 2013.
- [22] H. Kim, H. Lim, J. Jeong, H. Jo, and J. Lee, "Task-aware virtual machine scheduling for I/O performance," in *Proc. 5th Int. Conf. Virtual Execution Environ.*, 2009, pp. 101–110.
- [23] M. Lee, A. S. Krishnakumar, P. Krishnan, N. Singh, and S. Jaynik, "Supporting soft real-time tasks in the xen hypervisor," in *Proc. 6th Int. Conf. Virtual Execution Environ.*, 2010, pp. 97–108.
- [24] W. Long, "qspinlock: a 4-byte queue spinlock with PV support," May 2014. [Online]. Available: <https://lwn.net/Articles/597672/>
- [25] J. C. Mogul and A. Borg, "The effect of context switches on cache performance," in *Proc. 4th Int. Conf. Architectural Support Program. Lang. Operating Syst.*, 1991.
- [26] D. Ongaro, A. L. Cox, and S. Rixner, "Scheduling I/O in virtual machine monitors," in *Proc. 4th Int. Conf. Virtual Execution Environ.*, 2008, pp. 1–10.
- [27] J. Ouyang and J. R. Lange, "Preemptible ticket spinlocks: Improving consolidated performance in the cloud," in *Proc. 9th Int. Conf. Virtual Execution Environ.*, 2013.
- [28] J. Ouyang, J. R. Lange, and H. Zheng, "Shoot4U: Using VMM assists to optimize TLB operations on preempted vCPUs," in *Proc. 12th Int. Conf. Virtual Execution Environ.*, 2016, pp. 17–23.
- [29] G. Ren, E. Tune, T. Moseley, Y. Shi, S. Rus, and R. Hundt, "Google-wide profiling: A continuous profiling infrastructure for data centers," *IEEE Micro*, vol. 30, no. 4, Jul. 2010, Art. no. 6579. [Online]. Available: <https://doi.org/10.1109/MM.2010.68>
- [30] O. Sukwong and H. S. Kim, "Is co-scheduling too expensive for SMP VMs?" in *Proc. 6th Eur. Conf. Comput. Syst.*, 2011, pp. 257–272.
- [31] B. Teabe, V. Nitu, A. Tchana, and D. Hagimont, "The lock holder and the lock waiter pre-emption problems: Nip them in the bud using informed spinlocks (l-spinlock)," in *Proc. 12th Eur. Conf. Comput. Syst.*, 2017, pp. 286–297.
- [32] B. Teabe, A. Tchana, and D. Hagimont, "Application-specific quantum for multi-core platform scheduler," in *Proc. 11th Eur. Conf. Comput. Syst.*, 2016, Art. no. 3.
- [33] D. Tsafir, "The context-switch overhead inflicted by hardware interrupts (and the enigma of do-nothing loops)," in *Proc. Workshop Exp. Comput. Sci.*, 2007.
- [34] "VMware vSphere 4: The CPU scheduler in VMware ESX 4.1," *Technical Whitepaper*, VMware, 2010.
- [35] P. M. Wells, K. Chakraborty, and G. S. Sohi, "Hardware support for spin management in overcommitted virtual machines," in *Proc. 15th Int. Conf. Parallel Architectures Compilation Tech.*, 2006, pp. 124–133.
- [36] X. P. wiki, "Cpupools Howto," Jul. 2019. [Online]. Available: https://wiki.xen.org/wiki/Cpupools_Howto
- [37] S. Xi, J. Wilson, C. Lu, and C. Gill, "RT-Xen: Towards real-time hypervisor scheduling in Xen," in *Proc. Int. Conf. Embedded Softw.*, 2011, pp. 39–48.
- [38] C. Xu, S. Gamage, H. Lu, R. Kompella, and D. Xu, "vTurbo: Accelerating virtual machine I/O processing using designated turbo-sliced core," in *Proc. USENIX Conf. Annu. Tech. Conf.*, 2013.
- [39] C. Xu, S. Gamage, P. N. Rao, A. Kangarlou, R. R. Kompella, and D. Xu, "vSlicer: Latency-aware virtual machine scheduling via differentiated-frequency CPU slicing," in *Proc. 21st Int. Symp. High-Perform. Parallel Distrib. Comput.*, 2012.
- [40] J. Zebchuk, H. Cain, X. Tong, V. Srinivasan, and A. Moshovos, "RECAP: A region-based cure for the common cold (cache)," in *Proc. 19th Int. Symp. High Perform. Comput. Architecture*, 2013, pp. 83–94.
- [41] X. Zhang, E. Tune, R. Hagmann, R. Jnagal, V. Gokhale, and J. Wilkes, "CPI2: CPU performance isolation for shared compute clusters," in *Proc. 8th ACM Eur. Conf. Comput. Syst.*, 2013, pp. 379–391.



Taeklim Kim is currently working toward the BS degree with the Department of Software and Computer Engineering, Ajou University. His research interests include computer systems and architecture for cloud computing.



Chang Hyun Park received the BS degree in computer science and engineering from Hanyang University, in 2013, and the MS and PhD degrees in computer science from KAIST, in 2015 and 2019, respectively. He is a postdoctoral researcher with the Department of Information Technology, Uppsala University. His research interests include computer architecture and operating systems.



Jaehyuk Huh (Member, IEEE) received the BS degree in computer science from Seoul National University, and the MS and PhD degrees in computer science from the University of Texas at Austin. He is currently a professor of computer science at Korea Advanced Institute of Science & Technology. His research interests include computer architecture, parallel computing, virtualization, and system security.



Jeongseob Ahn received the BS degree in computer science and engineering from Dongguk University, in 2009, and the MS and PhD degrees in computer science from the Korea Advanced Institute of Science & Technology, in 2011 and 2015, respectively. He is currently an assistant professor with the Department of Software and Computer Engineering, Ajou University. His research interests include building high performance computer systems.

▷ For more information on this or any other computing topic, please visit our Digital Library at www.computer.org/csdl.